Multiplayer Blackjack

Software Design Specification

Revision History

Date	Revision	Description	Author
03/05/2026	1.0	Initial Version	Michael, Curtis, Emma

Table of Contents

Use Case Specifications **4**
UML Use Case Diagram **14**
UML Class Diagram **15**
UML Sequence Diagrams **16**

Use Case Specifications

Use Case ID: USER-01
Use Case Name: Register
Relevant Requirements:
- 3.1.4.1
- 3.1.5.1

Primary Actor: User (Player or Dealer)
Pre-conditions:
- There is no account currently logged in on the user's system.

Post-conditions:
- The user has created an account.
- The user is logged into the new account.

Basic Flow or Main Scenario:
1. The system prompts for an action (sign in, register as player, or register as dealer).
2. The user chooses to register (as a player or a dealer).
3. The system prompts for a username and password to create an account with.
4. The user enters a username and password.
5. The system validates the username and password.
6. An account (either player or dealer) is created and automatically signed in.

Extensions or Alternate Flows:
Extension 1: Username is Invalid
5.1. The system informs the user that the username is invalid (not available).
5.2. Process returns to step 3.
Extension 2: Password is Invalid
5.1. The system informs the user that the password is invalid (doesn't meet requirements).
5.2. Process returns to step 3.

Exceptions:
Exception 1: Invalid Input
Related Use Cases: N/A

Use Case ID: USER-02
Use Case Name: Log in
Relevant Requirements:
- 3.1.4.2
- 3.1.5.2

Primary Actor: User (Player or Dealer)
Pre-conditions:
- There is no account currently logged in on the user's system.
- The account exists.

Post-conditions:
- The user is logged in on their system.

Basic Flow or Main Scenario:
1. The system prompts for an action (sign in, register as player, or register as dealer).
2. The user chooses to sign in.
3. The system prompts for a username and password.
4. The user enters a username and password.
5. The system validates the username and password.
6. The account attached to the given username and password is signed in.

Extensions or Alternate Flows:
Extension 1: Username is Invalid
5.1. The system informs the user that the username is invalid (doesn't exist).
5.2. Process returns to step 3.

Extension 2: Password is Invalid
5.1. The system informs the user that the password is invalid (doesn't match username).
5.2. Process returns to step 3.

Exceptions:
Exception 1: Input Invalid
Related Use Cases:
- USER-01: Register

Use Case ID: PLAYER-01
Use Case Name: Check Balance
Relevant Requirements:
- 3.1.5.5
- 3.2.5

Primary Actor: Player
Pre-conditions:
- The player is logged in.

Post-conditions:
- The player's current balance is viewable by the player.

Basic Flow or Main Scenario:
1. The player is signed in.
2. The system prompts for an action (view account balance, add funds, cash out, join blackjack table, or log out).
3. The user chooses to view balance.
4. The system fetches the current balance attached to the signed in player account.
5. The system displays the account balance.

Extensions or Alternate Flows: N/A
Exceptions: N/A
Related Use Cases:
- USER-02: Log in

Use Case ID: PLAYER-02
Use Case Name: Deposit
Relevant Requirements:
- 3.1.5.6
- 3.2.5

Primary Actor: Player
Pre-conditions:
- The player is logged in.

Post-conditions:
- The player's current balance is updated including the new deposit.

Basic Flow or Main Scenario:
1. The player is signed in.
2. The system prompts for an action (view account balance, add funds, cash out, join blackjack table, or log out).
3. The user chooses to add funds.
4. The system fetches the current balance attached to the signed in player account.
5. The system prompts for the deposit amount.
6. The player enters the deposit amount.
7. The system verifies the user input.
8. The system adds the deposit amount to the player's account balance.
9. The system displays the new account balance.

Extensions or Alternate Flows:
Extension 1: Invalid deposit input
7.1. The system informs the user that the input is invalid.
7.2. Process returns to step 5.

Exceptions:
Exception 1: Input Invalid
Related Use Cases:
- USER-02: Log in

Use Case ID: PLAYER-03
Use Case Name: Withdraw
Relevant Requirements:
- 3.1.5.6
- 3.2.5

Primary Actor: Player
Pre-conditions:
- The player is logged in.

Post-conditions:
- The player's current balance is updated removing the withdrawal amount.

Basic Flow or Main Scenario:
1. The player is signed in.
2. The system prompts for an action (view account balance, add funds, cash out, join blackjack table, or log out).
3. The user chooses to cash out.
4. The system fetches the current balance attached to the signed in player account.
5. The system prompts for the withdrawal amount.
6. The player enters the withdrawal amount.
7. The system verifies the user input.
8. The system removes the withdrawal amount from the player's account balance.
9. The system displays the new account balance.

Extensions or Alternate Flows:
Extension 1: Invalid withdrawal input
- 7.1. The system informs the user that the input is invalid.
- 7.2. Process returns to step 5.

Exceptions:
Exception 1: Input Invalid
Related Use Cases:
- USER-02: Log in

Use Case ID: PLAYER-03
Use Case Name: “Join table”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.7
Primary Actor: Player or dealer
Pre-conditions: The user, whether it be a player or dealer, is logged into the program. The user also does not have a time out.
Post-conditions: The user is at a table. A table may be opened.
Basic Flow or Main Scenario:

1.The player is logged into the Multiplayer Blackjack program;
2. The player requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds an available table;
5. The player joins that table.

Extensions or Alternate Flows:
Extension 1: no table available

1.The player is logged into the Multiplayer Blackjack program;
2. The player requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds no available table, so it creates a new table;
5. The player joins the new table.

Alternate Flow 1: Dealer case

1.The dealer is logged into the Multiplayer Blackjack program;
2. The dealer requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds an available table;
5. The dealer joins that table.

Extension 2: Dealer case, no table available

1.The dealer is logged into the Multiplayer Blackjack program;
2. The dealer requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds no available table, so it creates a new table;
5. The player joins the new table.

Extension 3: Player timed out

1.The player is logged into the Multiplayer Blackjack program;
2. The player requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server informs the client that the player is timed out for some time;
5. The client UI displays that the player is timed out and how long until they are free to join again.

Exceptions: The player is timed out
Related Use Cases: USER-02: Log in, PLAYER-04: Bet

Use Case ID: PLAYER-04
Use Case Name: “Bet”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.5, 3.1.6
Primary Actor: Player
Pre-conditions: The player is at a table and is currently active in a blackjack game.
Post-conditions: The player has placed a bet, which is subtracted from their balance.
Basic Flow or Main Scenario:

1.The player is logged into the Multiplayer Blackjack program;
2.The player joins a table;
3. A game of blackjack begins;
4. The player is prompted by the server to place a bet;
5. The player chooses the chips they wish to bet with through the UI;
6. The server checks the player’s balance;
7. The player can afford the bet;
8. The bet is placed, with currency subtracted from the balance equal to the worth of the chips.

Extensions or Alternate Flows:
Extension 1: Invalid bet

1.The player is logged into the Multiplayer Blackjack program;
2.The player joins a table;
3. A game of blackjack begins;
4. The player is prompted by the server to place a bet;
5. The player chooses the chips they wish to bet with through the UI;
6. The server checks the player’s balance;
7. The player cannot afford the bet;
8. The server tells the client that the player cannot place the bet;
9. The process repeats from step 4;
10. If no bet is placed in time, a bet of the lowest amount is made automatically.

Exceptions: The player has insufficient balance or runs out of time.
Related Use Cases:

- USER-02: Log in
- USER-03: Join table
- USER-05: Hit
- USER-06: Stand

Use Case ID: USER-04
Use Case Name: “Leave table”
Relevant Requirements: 3.1.7
Primary Actor: Player or dealer
Pre-conditions: The user is at a table.
Post-conditions: The user is returned to the starting menu or will be returned after a game concludes.
Basic Flow or Main Scenario:

1.The player is logged into the Multiplayer Blackjack program;
2.The player is at a table;
3. The player requests from the client UI to leave the table;
4. The server removes the user from the table; 5. The player is back at the starting menu.

Extensions or Alternate Flows:
Alternate flow 1: Dealer case, game active

1. The dealer is logged into the Multiplayer Blackjack program;
2.The dealer is at a table;
3. The dealer requests from the client UI to leave the table;
4. There is a blackjack game active;
5. The server keeps a record that the dealer wants to leave;
6. After the game ends, the dealer automatically leaves the table;
7. The dealer is back at the starting menu.

Extension 1: Dealer case, no game active

1. The dealer is logged into the Multiplayer Blackjack program;
2. The dealer is at a table;
3. The dealer requests from the client UI to leave the table;
4. There is no blackjack game active;
5. The server removes the dealer from the table;
6. The dealer is back at the starting menu.

Exceptions: N/A
Related Use Cases:

- USER-02: Log in
- USER-03: Join table

Use Case ID: USER-05
Use Case Name: “Hit”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.4, 3.1.5, 3.1.7
Primary Actor: Player/Dealer
Pre-conditions: User is at a table, they are not busted, and it is currently their turn
Post-conditions: The user receives an additional card; if the total exceeds 21, the player busts And the turn ends
Basic Flow or Main Scenario:

1. The user requests an additional card from the Client UI.
2. The Client sends the “Hit” request to the server.
3. The server tells the Table class to execute the pullCard method from the CardDeck.
4. The system adds the card to the player’s hand and updates the hand total.
5. The system displays the new card and updates the total to the player.

Related Use Cases:

- USER-03: Join table
- USER-06: Stand

Use Case ID: USER-06
Use Case Name: “Stand”
Relevant Requirements: 3.1.3, 3.1.4, 3.1.5, 3.1.7
Primary Actor: Player/Dealer
Pre-conditions: The player is at a table, and it’s currently their turn
Post-conditions: The user is disconnected from the server and the application closes
Basic Flow or Main Scenario:

1. The user selects the “Stand” option from the Client UI.
2. The Client sends a standRequest.
3. The Server signals the Table class to trigger the nextTurn method.
4. The system locks the player’s UI for the remainder of the round.

Related Use Cases:

- USER-03: Join table
- USER-05: Hit

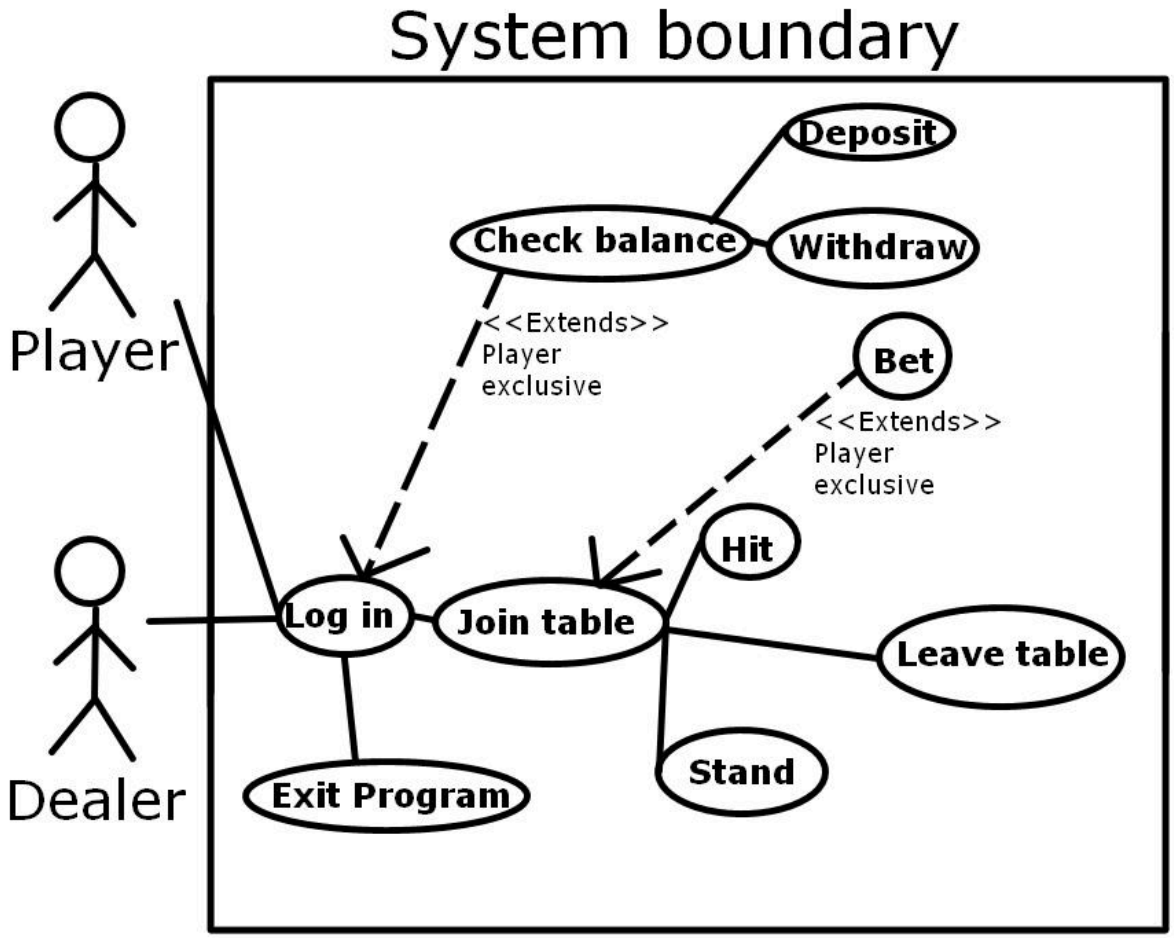
Use Case ID: USER-07
Use Case Name: “Exit Program”
Relevant Requirements: 3.1.3, 3.1.4, 3.1.5, 3.1.7
Primary Actor: Player/Dealer
Pre-conditions: The program is currently running.
Post-conditions: The user is disconnected from the server, and the application closes
Basic Flow or Main Scenario:

1. The user selects the “Exit” or “Close” option from the Client UI.
2. The Client sends a leaveRequest to the Server.
3. The Server removes the user from any active tables via the removeUserFromTable method.
4. The Server terminates the connection session.
5. The Client application closes.

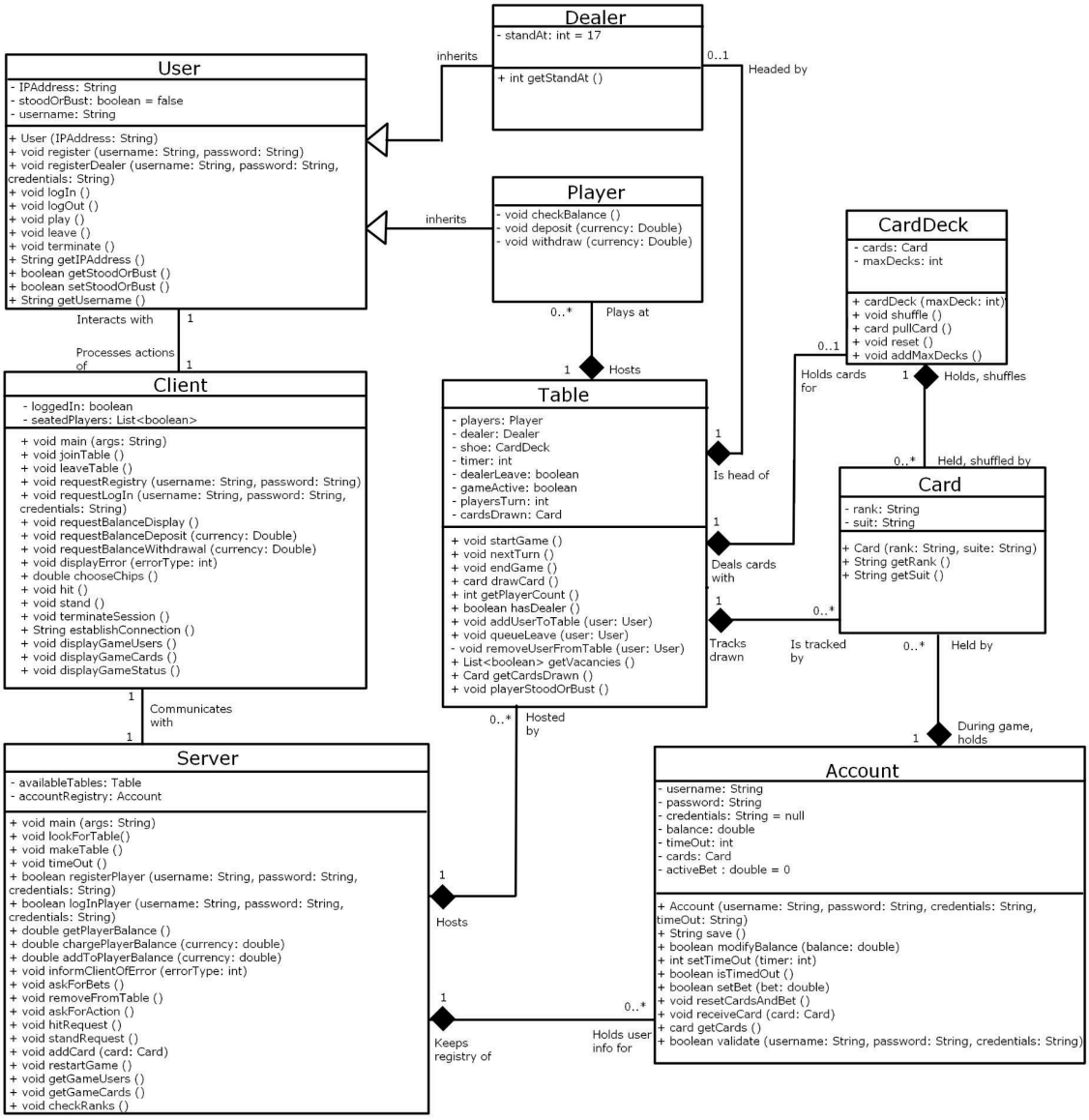
Related Use Cases:

- USER-02: Log in
- USER-04: Leave table

UML Use Case Diagram

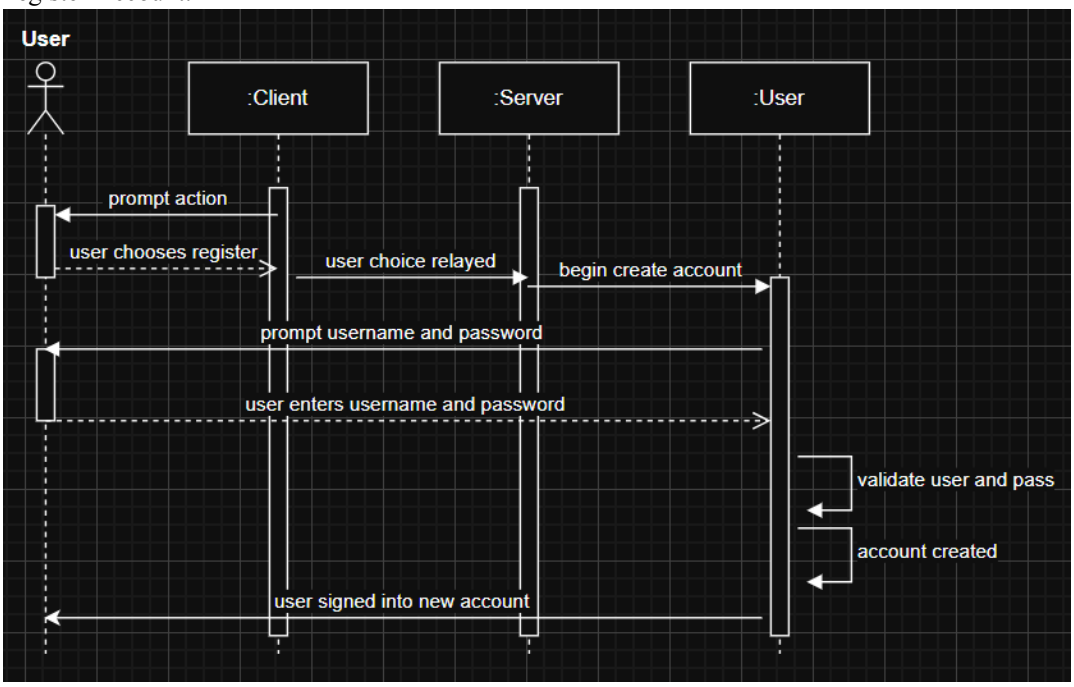


UML Class Diagram

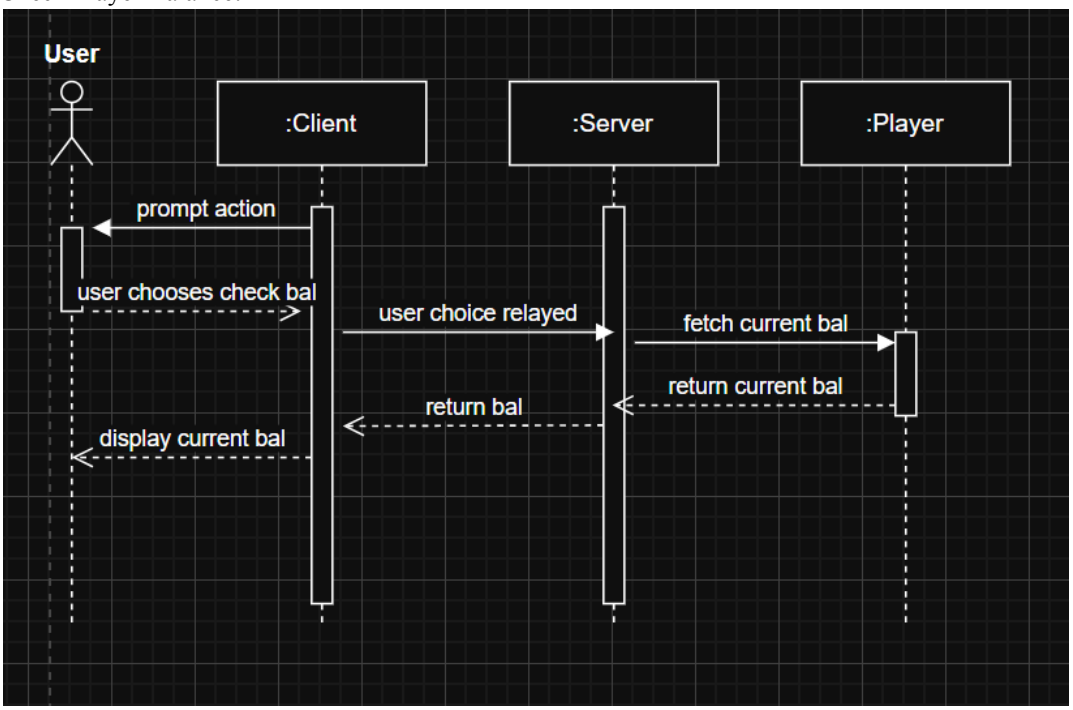


UML Sequence Diagrams

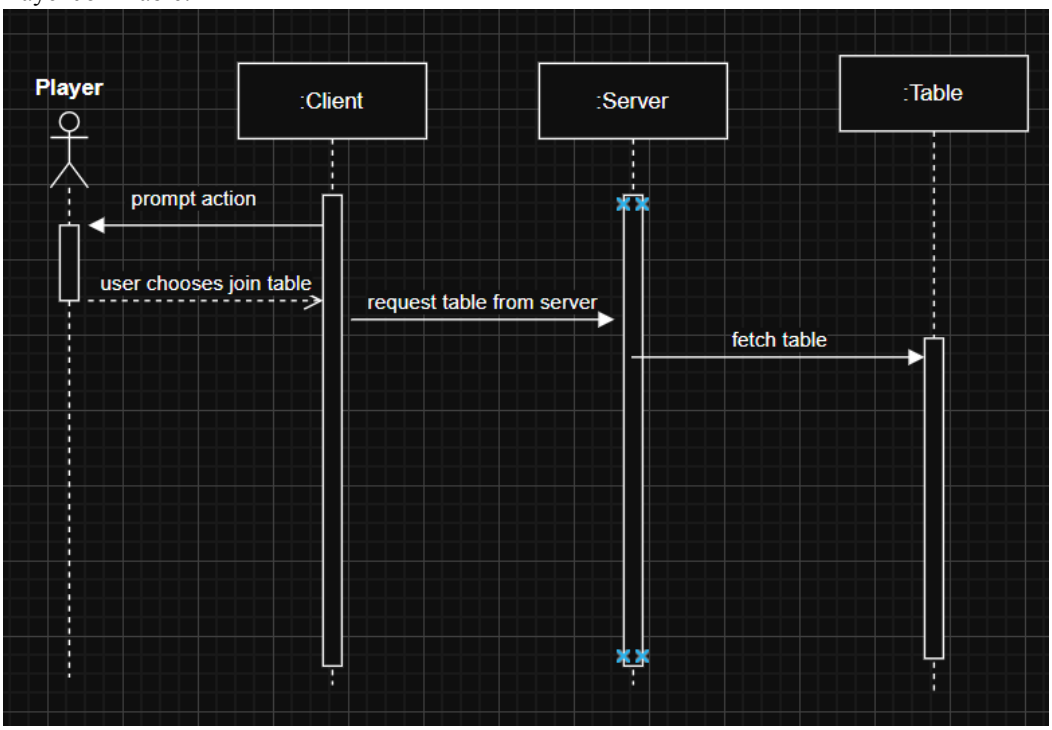
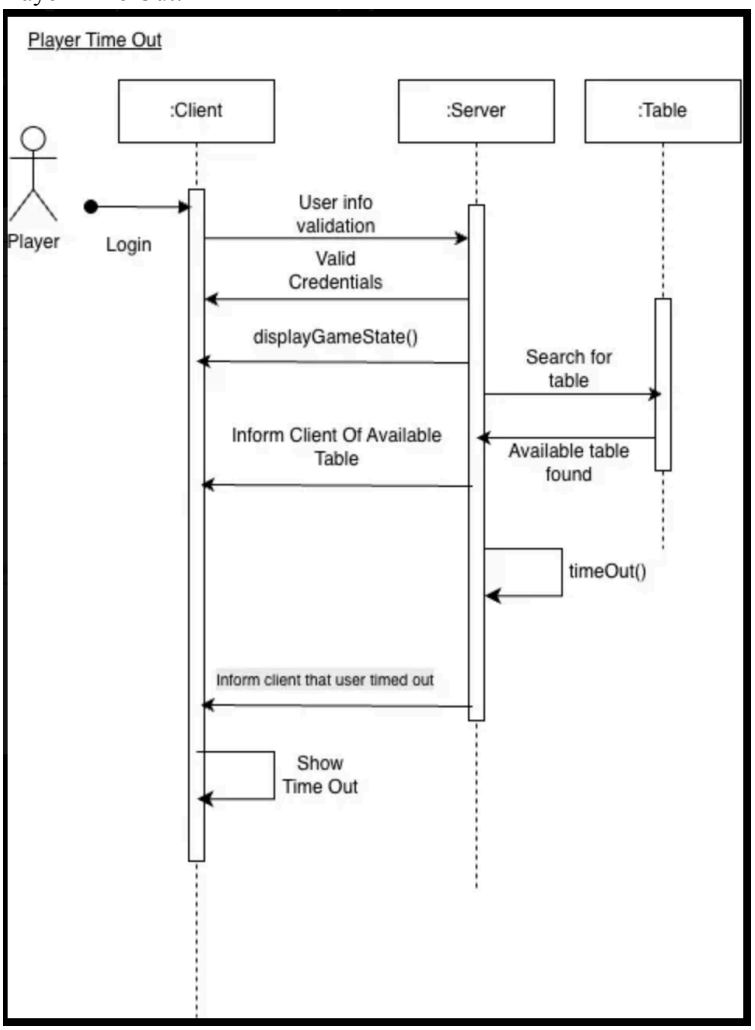
Register Account:



Check Player Balance:



Player Join Table:



Player Time Out: